

MENTORSHIP UPDATE (Latest Changes)

1. Development Environment Changed Completely

We decided that we will **not** develop directly on Windows anymore.

Instead, we will use the same workflow as professional AI engineers.

The new workflow is:

Windows



WSL2 (Windows Subsystem for Linux)



Ubuntu 24.04



VS Code (connected to WSL)



Python



Git



GitHub



Future: Docker, FastAPI, etc.

Windows will remain my operating system.

Ubuntu (inside WSL2) will become my complete programming environment.

All projects, Git commands, Python execution, package installation, and development should happen inside Ubuntu.

The goal is to become comfortable with Linux before graduation because almost every AI engineering environment uses Linux.

2. Learning Linux Is Now Part of the Roadmap

Linux is no longer something I will learn "later."

From now on I will learn Linux naturally while building projects.

Whenever possible, ChatGPT should teach me Linux commands as I use them instead of giving me an isolated Linux course.

For example:

- pwd
- ls
- cd
- mkdir
- touch
- cp
- mv
- rm
- cat
- grep
- find
- chmod
- git
- python3
- pip

should all become natural tools that I use daily.

The objective is that by graduation Linux feels normal instead of intimidating.

3. Project Name Changed

Originally we were building a project called:

Dataset Explorer

We decided that this was too limited.

The new long-term project is called:

DataLab AI

Dataset Explorer becomes only the **first module** inside DataLab AI.

This allows the project to continue growing for several years without needing to start over.

Future modules can include:

- Dataset Explorer
- Data Cleaner
- Visualization Studio
- Statistics Module
- Machine Learning Module
- Report Generator
- AI Assistant
- FastAPI Backend
- Web Dashboard

The project should evolve throughout my degree.

4. We Changed Our Engineering Philosophy

Originally the focus was on simply writing Python code.

Now the focus is:

Learn Software Engineering while learning Python.

Every feature should teach at least one engineering principle.

Examples:

- Separation of Concerns
- Modular Design
- Clean Architecture
- Meaningful Naming
- Documentation
- Git Workflow
- Error Handling
- Code Readability

The objective is to become an engineer, not simply someone who knows Python syntax.

5. We Changed the Mentorship Style

ChatGPT should no longer behave like a programming teacher.

Instead it should behave like a senior engineer reviewing the work of a junior engineer.

Whenever I finish a feature:

Instead of saying:

"Good job."

ChatGPT should ask questions like:

- Why did you design it this way?
- Could this function have one responsibility instead of two?
- What happens if the user enters invalid input?
- How would this scale in a larger application?
- How could the naming be improved?

ChatGPT should review my code honestly.

I do not want easy praise.

I want professional code reviews.

6. We Changed How Python Will Be Learned

We decided not to follow a traditional Python course.

Reason:

I already know C++.

I already understand programming fundamentals.

Instead I will learn Python through:

- Building projects
- Reading documentation
- Solving LeetCode in Python
- Using Python every day

ChatGPT should explain Python only when it naturally appears during development.

The project itself should become my Python course.

7. We Changed How Sessions Will Work

Every work session should roughly follow this structure:

Mission

↓

Small amount of theory

↓

Architecture discussion

↓

Implementation

↓

Testing

↓

Git commit

↓

Code review

↓

Reflection

Most of the time should be spent building.

Not watching tutorials.

8. Current Progress

At this point we have intentionally NOT written the first feature yet.

We spent our time making good engineering decisions before coding.

The next session should begin by configuring the development environment correctly and then building the project from the ground up.

9. Immediate Next Goal

The next coding session should begin with:

- Installing WSL2 (if not already installed).
- Installing Ubuntu.
- Configuring VS Code to work with WSL.
- Installing Git inside Ubuntu.
- Installing Python inside Ubuntu.
- Creating the DataLab AI project structure.
- Initializing Git.
- Creating a virtual environment.
- Installing the first dependencies.
- Making the initial commit.

Only after the development environment is ready should we begin implementing Version 0.1.

10. Version 0.1 Goal

The first working version of DataLab AI should be a simple terminal application.

The user should be able to:

- Start the application.
- Choose "Load Dataset."
- Enter the path to a CSV file.
- Load the dataset with pandas.
- Display:
 - Number of rows
 - Number of columns
 - Column names
- Handle invalid file paths without crashing.

This first feature should emphasize clean architecture, separation of concerns, and professional coding practices rather than advanced functionality.

11. One New Rule

We added an important rule for this mentorship:

We are building a product, not completing tasks.

Every feature should be designed so that it can remain part of the project years from now.

Nothing should feel like "throwaway code."

Every commit should make DataLab AI a better piece of software than it was the day before.